Manuel d'installation et de configuration - pili-pos v1.0.0

Ce manuel s'adresse au technicien installateur (= distributeur, nuagemagique.dev). Il décrit la mise en place complète d'un kiosque pili-pos chez un client HoReCa.

1. Matériel requis

Élément	Modèle / spécifications	Qté
Terminal POS	Penetek A3216 M82 ou Advantech USC-360J (Intel J6412, 8 Go RAM, 128 Go SSD, écran 15,6" 1920×1080 PCAP, IP65)	1
Boîtier FDM	FDM module2 (PRACSYS, certifié SCE 2.0) ou autre FDM certifié SCE 2.0	1
Imprimante thermique	Epson TM-m30III, raccordée au LAN (le POS imprime en TCP brut, port 9100 ; l'USB n'est pas pris en charge)	1
Tiroir-caisse	Standard, connexion RJ11 vers l'imprimante	1
Câble Ethernet	RJ45 Cat-5e ou supérieur	3 (POS, FDM, imprimante)
Switch / box internet	4 ports minimum	1
Clé USB d'installation/mise à jour	8 Go minimum - l'image kiosque complète (`pili-pos-kiosk-<profil>.img.xz`) y est écrite telle quelle (§2), pas un simple installeur FAT32	1
Onduleur (recommandé)	600 VA minimum	1

Puce TPM 2.0 obligatoire (fTPM Intel PTT intégré au J6412, ou puce dTPM discrète selon la carte mère) : le premier démarrage chiffre `/var` (base fiscale) en LUKS2 avec déverrouillage automatique lié à ce TPM (§3.4, `SECURITE_BD_FR.md` §3.6). Sans TPM2 activé en BIOS, le premier démarrage échoue à l'étape d'enrôlement TPM2 - vérifier et activer avant de commencer (§3.1).

2. Préparation de la clé USB d'installation

2.1 Télécharger l'image kiosque

L'image est fournie par le distributeur sous forme d'un fichier `pili-pos-kiosk-vX.Y.Z.img.xz`. Sur la machine du distributeur :

```
xz -d pili-pos-kiosk-vX.Y.Z.img.xz
```

2.2 Écrire l'image sur la clé USB

Attention : **L'opération efface toute la clé USB.**

Sur Linux/macOS :

```
# Identifier la clé (souvent /dev/sdX ou /dev/diskN)
lsblk        # Linux
diskutil list # macOS
```

```
# Écrire l'image (remplacer /dev/sdX par le bon device)
sudo dd if=pili-pos-kiosk-vX.Y.Z.img of=/dev/sdX bs=4M status=progress conv=fsync
```

Sur Windows : utiliser **Rufus** ou **Balena Etcher** avec l'image.

3. Installation sur le terminal POS

3.1 Configuration BIOS

À la première mise sous tension du terminal :

1. Entrer dans le BIOS (touche `Del` ou `F2` selon le modèle).
2. Activer **Boot from USB**.
3. **Activer le TPM 2.0** (souvent affiché "Intel PTT" ou "fTPM" sur le J6412) - obligatoire pour le chiffrement automatique de `/var` au premier démarrage (§3.4, §1). Sans cela, `pili-provision` échoue à l'enrôlement TPM2 et le terminal reste bloqué avant d'atteindre l'écran de connexion.
4. **Laisser Secure Boot désactivé** (le paramètre par défaut sur la plupart de ces cartes mères) : l'image ne signe pas encore son UKI (Unified Kernel Image) contre une clé Secure Boot - l'activer empêcherait le terminal de démarrer. Voir `SECURITE_BD_FR.md` §3.6 pour ce que cela implique pour le scellement TPM2.
5. **Définir un mot de passe BIOS** (à conserver précieusement - le client n'en a pas besoin) et verrouiller l'ordre de boot sur le disque interne pour l'usage normal (USB seulement au besoin, via le mot de passe BIOS).
6. Sauvegarder et redémarrer.

3.2 Boot sur la clé USB

1. Brancher la clé USB préparée (image kiosque écrite telle quelle, §2 - elle contient déjà les deux entrées de menu suivantes, ajoutées par `deploy/image/build.sh` en dupliquant l'UKI construit par mkosi avec une ligne de commande différente).
2. Démarrer le terminal - il doit booter sur la clé (menu `systemd-boot`).
3. Choisir l'entrée **"Install"** pour une installation neuve, ou **"Update"** pour mettre à jour un terminal déjà en service sans perdre `/var` (base fiscale + sauvegardes) - voir §6.

3.3 Procédure d'installation

Le service `pili-installer.service`, actif uniquement quand le noyau a démarré avec `pili.installer=install` (l'entrée "Install" du menu), fait le travail :

1. Détection du disque interne (le premier disque autre que la clé USB elle-même).
2. **Install** : réplication complète de la table de partitions de la clé (`sgdisk --replicate` + `-G` pour de nouveaux UUID) puis copie partition par partition (ESP + racine ; la partition `pili-data` copiée est vide/non formatée à ce stade) vers le disque interne. **Update** : seules les partitions ESP et racine sont recopiées (dd partition à partition) ; la partition `pili-data` déjà présente sur le disque interne (avec son contenu chiffré) n'est **jamais touchée** - c'est ce qui préserve `pos.db` d'une mise à jour à l'autre. Les mots de passe `root` et `spf` de l'ancienne racine sont reportés sur la nouvelle.
3. Le disque interne contient maintenant Debian 13 minimal + pili-pos
   - Cage + RustDesk (extrait) + les outils de provisioning, en lecture seule pour la racine - mais rien n'est encore déchiffré ni configuré : cette étape ne fait que poser les partitions ESP/racine sur le disque interne.

Durée approximative : 3 à 6 minutes (copie de partitions brutes, beaucoup plus rapide que l'ancienne installation "à chaud" sur un Debian nu).

3.4 Premier démarrage - chiffrement, mots de passe, clé de récupération

Retirer la clé USB et redémarrer. Le terminal doit :

1. Démarrer sur le disque interne (systemd-boot → UKI).
2. `systemd-repart` finalise la partition `pili-data` (taille = tout l'espace libre restant sur le disque).
3. `pili-provision.service` s'exécute automatiquement, **une seule fois** (il ne se relance jamais aux démarrages suivants) :
   - Formate `pili-data` en LUKS2 avec une clé temporaire.
   - Enrôle le TPM2 (déverrouillage automatique, silencieux, à chaque démarrage normal) et une **clé de récupération**.
   - Supprime la clé temporaire.
   - Formate le volume déchiffré en ext4, le monte sur `/var`.
   - Copie la base et le `pos.json` d'usine, y inscrit le `device_id` (adresse MAC de la carte réseau du terminal, en minuscules - voir §4.2 ; jamais `DEV001` ni une valeur libre).
   - Génère un mot de passe `root` aléatoire (et, sur l'image d'admission, un mot de passe `spf`), les affiche **une seule fois** à l'écran avec la clé de récupération LUKS, puis attend une pression sur Entrée.
4. **Inscrire immédiatement** le mot de passe `root` (et `spf` le cas échéant) ainsi que la clé de récupération LUKS dans le carnet du distributeur (jamais communiqués au client) - ces valeurs ne sont **plus jamais réaffichées** et ne sont conservées nulle part sur le disque après cet écran.
5. Une fois Entrée pressée, le terminal poursuit son démarrage : Cage Wayland en plein écran, puis pili-pos en mode kiosque.
6. Écran de connexion affiché. Tant que les champs propres à l'établissement de `pos.json` (§4.2) ont encore la valeur du gabarit d'usine (`pos_id`, `restaurant_vat`, `restaurant_est_no`), un bandeau rouge s'affiche en bas de l'écran : "Configuration invalide : le FDM refusera les tickets. Appelez l'installateur.", suivi de la liste des champs en cause. Il disparaît quand `pos.json` est valide et que le service a redémarré (§4.2, "Vérification").

Durée approximative de cette étape : 1 à 3 minutes (l'enrôlement TPM2 est l'opération la plus lente).

Coupure de courant pendant cette étape : si le terminal est mis hors tension entre l'enrôlement TPM2 et le montage de `/var`, `pili-provision` détecte au redémarrage suivant un volume LUKS2 déjà enrôlé TPM2 mais non monté, et laisse le système le déverrouiller et le monter. Il ne génère pas de nouveaux mots de passe et n'affiche plus l'écran des mots de passe. Si le volume est LUKS2 mais **sans** jeton TPM2 (coupure encore plus tôt), le script s'arrête, affiche qu'une reprise manuelle est nécessaire depuis la console de maintenance (§4.1) et ne reprend pas seul. Aucune reprise automatique n'existe dans ces deux cas pour les mots de passe et la clé de récupération : si l'écran des mots de passe n'a pas été validé, refaire une installation complète (entrée "Install", §3.3) avant toute mise en service.

4. Configuration initiale

4.1 Accès maintenance

Le terminal n'a ni serveur SSH ni clavier branché : l'écran tactile n'affiche que pili-pos (console `tty1`). La seule console de maintenance est une console locale sur `tty2`, protégée par le mot de passe `root`. Pour la maintenance, le distributeur, sur place :

1. Branche un clavier USB sur le terminal.
2. Presse `Ctrl + Alt + F2` (séquence cachée non accessible à l'opérateur).
3. Se connecte en `root` avec le mot de passe noté à l'installation.

4.2 Fichier de configuration `/etc/pili-pos/pos.json`

Ce fichier **existe déjà** à ce stade : `pili-provision.service` l'a copié depuis le gabarit d'usine au premier démarrage (§3.4) et y a déjà inscrit le `device_id` réel du terminal. `/etc/pili-pos` est un lien symbolique vers `/var/lib/pili-etc/pili-pos` (partition chiffrée LUKS2, §3.4) - il ne redevient lisible qu'après déverrouillage, donc uniquement une fois le terminal démarré. Reste à compléter les champs propres à l'établissement.

Éditer avec `vi /etc/pili-pos/pos.json` (l'éditeur `vim-tiny` est installé sur l'image précisément pour cette étape) :

```
{
  "pos_id": "CFOD0061234567",
  "terminal_id": "POS1",
  "device_id": "00:1b:44:11:3a:b7",
  "restaurant_name": "Restaurant Exemple",
  "restaurant_legal_name": "Exemple Horeca SRL",
  "restaurant_address": "Rue de l'Exemple 1, 4000 Liège",
  "restaurant_vat": "BE0499999960",
  "restaurant_est_no": "2123456791",
  "fdm_url": "https://192.168.1.20/graphql/",
  "fdm_token_file": "/etc/pili-pos/secrets/fdm.token",
  "fdm_ca_file": "/etc/pili-pos/fdm/ca.crt",
  "fdm_client_cert_file": "/etc/pili-pos/fdm/client.crt",
  "fdm_client_key_file": "/etc/pili-pos/fdm/client.key",
  "printer_address": "192.168.1.30:9100",
  "kitchen_printer_address": "192.168.1.31:9100",
  "training_mode": false,
  "day_end_hour": 5,
  "table_count": 20,
  "language": "fr",
  "backup_dir": "/var/lib/pili-pos/backups"
}
```

Champs :

Champ	Description	Source
`pos_id`	Identifiant POS : 14 caractères, lettres majuscules ou chiffres (commence par `C`). Attribué par le SPF Finances à la certification.	SPF Finances
`terminal_id`	ID du terminal physique. Libre, en général `POS1`, `POS2`…	Distributeur
`device_id`	Adresse MAC de la carte réseau du terminal (minuscules, séparée par `:`), imprimée sur chaque ticket (manuel §VI, ligne “Appareil: ”). Renseignée **automatiquement** par `pili-provision` au premier démarrage (interface de la route par défaut, sinon la première interface Ethernet non-`lo` - voir `deploy/provision/provision.sh::detect_device_id`) ; ne pas modifier à la main sauf changement matériel de la carte réseau.	Automatique (terminal)
`restaurant_name`	Nom commercial du restaurant tel qu’apparaît sur le ticket	Client
`restaurant_legal_name`	Facultatif. Dénomination légale de la société exploitante, imprimée en bas du ticket (« Dénomination légale : »), avec l’adresse et le numéro de TVA. Absent : le nom commercial est imprimé.	Client
`restaurant_address`	Adresse complète	Client

Champ	Description	Source
`restaurant_vat`	Numéro de TVA belge : `BE` suivi de 10 chiffres commençant par 0 ou 1, valide modulo 97 (ex. `BE0499999960`)	Client
`restaurant_est_no`	Numéro d'unité d'établissement : 10 chiffres commençant par 2 à 8, valide modulo 97 (ex. `2123456791`)	BCE / client
`fdm_url`	URL GraphQL du FDM sur le LAN. FDM module2, configuration retenue : `https://<IP du FDM>/graphql/` (port 443, mTLS, voir §7.3) ; `http://<IP du FDM>/graphql/` (port 80) reste possible. Le gabarit d'usine contient `http://192.168.1.20/graphql` : à remplacer.	Configuration FDM
`fdm_token_file`	Fichier contenant le seul jeton d'accès (Bearer) du FDM (le jeton se lit sur la page de paramètres du FDM ou dans le portail module2) ; jamais le jeton lui-même dans `pos.json`. Un FDM module2 l'exige ; absent seulement pour un FDM qui n'en demande pas.	Configuration FDM
`fdm_ca_file`	Requis si `fdm_url` est en `https://`. Autorité qui signe le certificat du FDM (PEM ; `ca.crt` de module2).	Configuration FDM
`fdm_client_cert_file`, `fdm_client_key_file`	Ensemble ou pas du tout. Certificat client et sa clé (PEM ; `client.crt` et `client.key` de module2) pour un FDM en mTLS. Un fichier illisible s'affiche dans le bandeau des problèmes de configuration.	Configuration FDM
`printer_address`	IP:port de l'imprimante sur le LAN (port 9100 par défaut)	Configuration imprimante
`kitchen_printer_address`	Facultatif. IP:port de l'imprimante du bar ou de la cuisine, qui reçoit le « BON DE COMMANDE » (sans prix, avec la mention « ceci n'est pas un ticket de caisse TVA valable ») des lignes que l'écran de commande envoie en cuisine (§4.6, `MANUEL_UTILISATEUR_FR.md` §6.1). Absent : le bon sort sur `printer_address`. Le voyant "Cuisine" de l'en-tête reste gris tant qu'il est absent.	Configuration imprimante

Champ	Description	Source
`training_mode`	`false` en production. À NE JAMAIS laisser `true` après installation. C'est la valeur de départ : un Gérant peut changer le mode dans "Config" (§4.7).	Distributeur
`hubrise_enabled`	Facultatif, `false` par défaut. `true` : les commandes en ligne (plateformes de livraison, webshop, par HubRise) sont activées dès le démarrage. Un Gérant peut ensuite les activer ou les couper dans "Config", panneau "Intégrations" ; chaque changement est journalisé et le dernier l'emporte. Voir `COMMANDES_EN_LIGNE_FR.md`.	Client
`peppol`	Facultatif. Envoi des factures B2B sur le réseau Peppol (`FACTURATION_PEPPOL_FR.md`, `DECISION_PEPPOL_AP_FR.md`) : `provider` (`storecove` obligatoire en production), `base_url`, `api_key_file` (fichier contenant la seule clé de l'API), `legal_entity_id`, `sender_id` (`0208:` et le numéro d'entreprise à 10 chiffres), `supplier_street`, `supplier_postal_code`, `supplier_city`, `supplier_country` (`BE` par défaut). Sans cette section, une facture à un client étranger est établie et imprimée, et une facture à un client dont le numéro de TVA commence par `BE` est refusée (`invoice_peppol_required`).	Client
`table_count`	Nombre de tables du plan de salle, numérotées à partir de 1 ; défaut `20`. Ignoré quand `tables` est présent.	Client

Champ	Description	Source
`tables`	Facultatif. Numéros de table du plan de salle, en nombres et plages séparés par des virgules (ex. `"1–12, 20–25"`) : de 1 à 999, 200 tables au plus. Absent : les tables 1 à `table_count`. C'est la valeur de départ : un Gérant peut changer le plan dans "Config", panneau "Plan de salle" ; chaque changement est journalisé et le dernier l'emporte (une nouvelle valeur de `pos.json`, lue au démarrage, compte comme un changement). Une valeur invalide est signalée par `--check-config`.	Client
`day_end_hour`	Heure (0 à 12, heure belge) à laquelle la journée commerciale se termine ; défaut `5`. Un service qui dépasse minuit reste sur sa journée jusqu'à cette heure. Une journée restée ouverte au-delà est clôturée par un rapport Z automatique (FAQ producteurs Q501).	Client
`language`	Langue par défaut de l'interface : `fr` / `nl` / `de` / `en` / `zh`. Les mentions fiscales imprimées suivent cette clé, restreinte aux langues nationales (fr/nl/de) - toute autre valeur émet le ticket en FR.	Client
`backup_dir`	Répertoire des sauvegardes (défaut : `/var/lib/pili-pos/backups`)	Distributeur

Vérification Le FDM refuse tout événement dont l'identification est mal formée (description détaillée POS v2.2 §2.2.2). Après l'édition, vérifier :

```
/opt/pili-pos/pili-pos --check-config
```

La commande lit `pos.json` comme le POS et affiche `configuration OK`, ou la liste des champs à corriger (code de sortie 1) : format du `pos_id`, numéros de TVA et d'établissement (longueur, premier chiffre, modulo 97), champs vides, URL du FDM, langue, clé `tables`, bloc `digital_ticket`. Tant qu'un champ est refusé, le POS le signale aussi dans un bandeau rouge au bas de l'écran et dans le journal (`journalctl -u pili-pos`).

Langue du ticket La clé `language` détermine la langue des mentions fiscales imprimées. Elle est fixée à l'installation d'après la **région linguistique de l'établissement** où le POS est exploité (manuel de certification §VI, p. 99) - ni d'après le siège du distributeur, ni d'après la langue de l'interface souhaitée par le personnel :

Région linguistique de l'établissement	`language`
Région de langue française	`fr`
Région de langue néerlandaise	`nl`
Région de langue allemande	`de`
Région bilingue de Bruxelles-Capitale	`fr` ou `nl`

Région linguistique de l'établissement	`language`

Exemple : un établissement à 4000 Liège (région de langue française) → `"language": "fr"`. Voir `GARANTIE_MAJ_LANGUES_FR.md` §3.2.

Ticket numérique par e-mail (optionnel) Sans bloc `digital_ticket`, le POS n'émet que des tickets papier et n'affiche pas le choix du support. Pour proposer le ticket par e-mail (manuel de certification §III.H), ajouter à `pos.json` :

```
"digital_ticket": {
  "project_id": "<identifiant du projet Scaleway>",
  "from_email": "tickets@<domaine vérifié>",
  "from_name": "<nom du restaurant>",
  "api_key_file": "/etc/pili-pos/secrets/tem.key",
  "allow_digital_only": false
}
```

Champ	Description
`project_id`	Projet Scaleway du service Transactional Email.
`from_email`	Adresse d'expédition, sur un domaine vérifié dans Scaleway.
`from_name`	Nom affiché comme expéditeur (en général le nom du restaurant).
`api_key_file`	Fichier contenant la seule clé secrète de l'API, jamais la clé elle-même dans `pos.json`.
`allow_digital_only`	`true` : l'opérateur peut choisir « E-mail » seul, sans papier. `false` (défaut) : seuls « Papier » et « Les deux » sont proposés.
`region`	Facultatif, `fr-par` par défaut.

Installer la clé, lisible par le seul service POS :

```
install -d -m 750 -o root -g pos /etc/pili-pos/secrets
(umask 027; cat > /etc/pili-pos/secrets/tem.key)   # coller la clé, Entrée, puis Ctrl+D
chown root:pos /etc/pili-pos/secrets/tem.key
systemctl restart pili-pos
```

Le seul trafic sortant ajouté est HTTPS (443) vers `api.scaleway.com`. Si la clé est absente ou illisible, le POS le consigne dans son journal et reste en papier seul. Si le service ne répond pas, le choix « E-mail » seul devient « Les deux » : le client reçoit le ticket papier tout de suite et l'e-mail dès le retour du service (`TICKET_NUMERIQUE_FR.md`).

Commandes en ligne (HubRise). Sans bloc `hubrise`, la caisse ne peut pas se connecter à HubRise (« HubRise n'est pas configuré sur cette caisse »). Pour un établissement qui reçoit des commandes en ligne (`COMMANDES_EN_LIGNE_FR.md`), ajouter à `pos.json` le client OAuth HubRise fourni par le distributeur :

```
"hubrise_enabled": true,
"hubrise": {
  "client_id": "<identifiant>.clients.hubrise.com",
  "client_secret_file": "/etc/pili-pos/secrets/hubrise.secret"
}
```

Champ	Description
`client_id`	Identifiant du client OAuth HubRise de la caisse.
`client_secret_file`	Fichier contenant le seul secret du client, jamais le secret lui-même dans `pos.json`.
`oauth_base_url`	Facultatif, `https://manager.hubrise.com/oauth2/v1` par défaut.
`api_base_url`	Facultatif, `https://api.hubrise.com/v1` par défaut.
`redirect_uri`	Facultatif, `urn:ietf:wg:oauth:2.0:oob` par défaut : HubRise affiche un code à recopier dans la caisse.
`scope`	Facultatif, `location[orders.write,customer_list.read,catalog.read]` par défaut.

Installer le secret comme la clé du ticket numérique :

```
(umask 027; cat > /etc/pili-pos/secrets/hubrise.secret)   # coller le secret, Entrée, puis Ctrl+D
chown root:pos /etc/pili-pos/secrets/hubrise.secret
pili-pos --check-config        # signale un secret absent ou vide
systemctl restart pili-pos
```

Puis, connecté en Gérant : bouton “Config”, panneau “Intégrations” (HubRise est “Activé” si `hubrise_enabled` vaut `true`, sinon “Activer”), puis le panneau “HubRise” : « Se connecter à HubRise ». La caisse n’a pas de navigateur : elle affiche un QR code (et le lien en clair). Le scanner avec un téléphone, se connecter à HubRise, choisir l’établissement, puis taper dans la caisse le code affiché par HubRise (“Code d’autorisation”) et toucher « Valider la connexion ». Le trafic sortant ajouté est HTTPS (443) vers `manager.hubrise.com` et `api.hubrise.com`.

Facturation B2B et Peppol. La caisse établit des factures à des clients professionnels étrangers sans aucun réglage (`FACTURATION_PEPPOL_FR.md`). Pour facturer un client belge, qui doit être facturé sur le réseau Peppol, ajouter à `pos.json` la section du point d’accès, après avoir ouvert un compte chez le fournisseur (`DECISION_PEPPOL_AP_FR.md`) :

```
"peppol": {
  "provider": "storecove",
  "base_url": "https://api.storecove.com",
  "api_key_file": "/etc/pili-pos/secrets/peppol.key",
  "legal_entity_id": 123456,
  "sender_id": "0208:0123456789",
  "supplier_street": "Rue de l'Exemple 1",
  "supplier_postal_code": "4000",
  "supplier_city": "Liège"
}
```

Champ	Description
`provider`	`storecove` (obligatoire en production). `logging` journalise le document sans rien envoyer : réservé au développement, refusé par une version de production.
`base_url`, `api_key_file`, `legal_entity_id`	Requis pour `storecove` : adresse de l’API, fichier contenant la seule clé (jamais la clé dans `pos.json`), entité légale du fournisseur.
`sender_id`	Identifiant Peppol du vendeur : `0208:` et le numéro d’entreprise à 10 chiffres (valide modulo 97).

Champ	Description
`supplier_street`, `supplier_postal_code`, `supplier_city`	Adresse du vendeur dans le document.
`supplier_country`	Facultatif, `BE` par défaut (deux lettres majuscules).

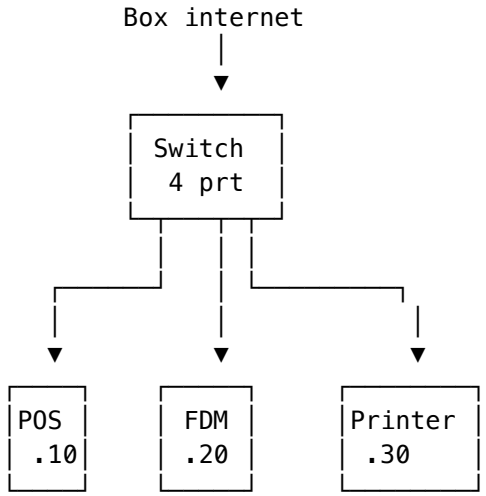
Installer la clé comme celle du ticket numérique :

```
(umask 027; cat > /etc/pili-pos/secrets/peppol.key)   # coller la clé, Entrée, puis Ctrl+D
chown root:pos /etc/pili-pos/secrets/peppol.key
pili-pos --check-config        # signale chaque réglage invalide : code config_peppol et nom du r
systemctl restart pili-pos
```

Sans cette section, une facture est établie et imprimée mais n'est pas envoyée : elle fait partie de la configuration d'une installation qui facture des clients professionnels belges. Le trafic sortant ajouté est HTTPS (443) vers l'adresse `base_url` (`api.storecove.com`). Le gérant suit les envois dans « Config », panneau « Peppol ».

4.3 Configuration réseau LAN

Le POS et le FDM doivent être sur le **même LAN privé**. Schéma type :



Adresses IP statiques recommandées (subnet `192.168.1.0/24`) : - POS : `192.168.1.10` - FDM : `192.168.1.20`, HTTPS port 443 (§4.2 et §7.3) - Imprimante : `192.168.1.30:9100`

(Réservations DHCP sur le routeur, cf. `SCHEMA_ASSEMBLAGE_FR.md` §4.) - Box : `192.168.1.1`

4.4 Configuration utilisateurs

Créer les utilisateurs initiaux dans la base SQLite :

```
sqlite3 /var/lib/pili-pos/pos.db
```

```
-- Gérant
INSERT INTO users (insz, name, role, active)
VALUES ('XXXXXXXXXXX', 'Bruno Moyse', 'admin', 1);

-- Serveur
INSERT INTO users (insz, name, role, active)
VALUES ('YYYYYYYYYYY', 'Marie Dupont', 'cashier', 1);

-- Apprenti
```

```
INSERT INTO users (insz, name, role, active)
VALUES ('ZZZZZZZZZZZ', 'Pierre Tran', 'waiter', 1);
```

Aucun PIN n'est saisi ici. Le Gérant choisit le sien à sa première connexion : sa tuile ouvre "Choisissez votre PIN" (un Gérant ne peut pas se connecter sans PIN). Le technicien reste présent jusqu'à ce choix : tant qu'aucun Gérant n'a de PIN, la première personne qui touche la tuile le choisit. Ce choix à l'écran de connexion n'existe que pour le premier Gérant ; un deuxième Gérant reçoit son PIN du premier, dans "Config", confirmé par le PIN de ce dernier. Le Serveur et l'Apprenti fixent le leur en touchant leur nom sur l'écran de commande. Ne pas créer l'INSZ `00000000029` : c'est l'identité système, qui ne se connecte jamais.

(Voir `MANUEL_PROGRAMMATION_FR.md` pour la création/modification de menus, modificateurs etc.)

4.5 Synchronisation horaire

Vérifier que `chrony` est actif :

```
systemctl status chrony
chronyc tracking
```

L'horloge **doit être à la seconde près**. Une dérive crée des ennuis à la signature FDM (qui exige un `posDateTime` cohérent).

4.6 Test du flux complet

Avant de quitter, effectuer un test de bout en bout :

1. Redémarrer le terminal (`reboot`).
2. Se connecter avec un INSZ valide.
3. Vérifier que le pointage S signe correctement.
4. Ouvrir une table, ajouter un item.
5. Encaisser en espèces - le ticket doit s'imprimer avec QR code et le tiroir s'ouvrir.
6. Inspecter `fiscal_events` :

```
SELECT id, event_type, sequence_number, short_signature
FROM fiscal_events ORDER BY id DESC LIMIT 5;
```

Trois lignes minimum : un S (work-in), un N (la vente), idéalement un P si la commande de table a été envoyée en cuisine ("Envoyer en cuisine" : le bon de commande s'imprime sur l'imprimante de la cuisine, ou sur celle des tickets si `kitchen_printer_address` est absent).

7. Tirer un rapport X pour validation finale ("Rapports", onglet "Journée", "Rapport X").

Terminal de paiement par carte relié (facultatif). Sans bloc `payment_terminal`, rien ne change : la carte est payée sur un terminal de paiement autonome, que l'opérateur saisit à la caisse (méthode `MANUAL`, type `CARD_DEBIT`). Avec ce bloc, la caisse envoie le montant au terminal et enregistre sa réponse (ligne de paiement `AUTOMATIC`, `MANUEL_UTILISATEUR_FR.md` §7.2). Trois protocoles sont pris en charge, chacun d'après une spécification publique ; `kind` choisit l'un d'eux :

`kind`	Protocole	Chemin
`worldline_terminal_api`	Worldline Terminal API (nexo Retailer, JSON sur HTTPS)	La caisse parle au nuage Worldline, qui joint le terminal (terminal Android Worldline avec l'application "Terminal API")
`ccv_payment_api`	CCV Payment API, méthode `terminal` (JSON sur HTTPS)	La caisse parle au nuage CCV, qui joint le terminal
`ccv_opi`	OPI (Open Payment Initiative, XML sur TCP)	Directement sur le LAN du restaurant, sans nuage

Le terminal Worldline C-TEP (gamme Yomani / Yoximo) n'est pas pris en charge : son protocole est fermé (bibliothèque et certification Worldline).

```
"payment_terminal": {
  "kind": "worldline_terminal_api",
  "merchant_id": "<UMID>",
  "terminal_id": "<UTID>",
  "integrator_id": "<identifiant d'intégrateur>",
  "api_key_file": "/etc/pili-pos/secrets/terminal.key"
}
```

Champ	Description
`kind`	`worldline_terminal_api`, `ccv_payment_api` ou `ccv_opi`. Obligatoire.
`base_url`	Types nuage seulement. Absent : l'adresse de production du fournisseur. À renseigner pour l'environnement de test du fournisseur ou pour un relais (voir plus bas).
`api_key_file`	Types nuage seulement. Fichier contenant la seule clé d'API, jamais la clé elle-même dans `pos.json`. Absent seulement quand `base_url` est un relais qui authentifie la caisse lui-même.
`merchant_id`, `terminal_id`, `integrator_id`	Worldline, tous trois obligatoires : identifiant du commerçant (UMID), du terminal (UTID) et de l'intégrateur que Worldline délivre au producteur.
`terminal_id`, `management_system_id`	CCV Payment API, tous deux obligatoires : identifiant du terminal attribué par CCV et identifiant du système de gestion du terminal.
`access_protocol`	CCV Payment API, facultatif : protocole d'accès du terminal, `OPI_NL` si absent.
`address`	OPI, obligatoire : `hôte:port` du terminal sur le LAN (canal 0, en général le port 4100).
`device_port`	OPI, facultatif : port d'écoute de la caisse pour le canal 1 du terminal (affichage et impression), 4102 si absent.
`terminal_id`	OPI, facultatif : identifiant de poste envoyé au terminal.
`timeout_seconds`	Facultatif, 30 à 600, 180 si absent : attente la plus longue de la réponse du terminal pour un paiement. Passé ce délai, le résultat est redemandé (statut inconnu, voir `MANUEL_UTILISATEUR_FR.md` §7.2).

Règle CCV : la clé d'API CCV ne doit pas se trouver sur un appareil du restaurant. Pour `ccv_payment_api`, renseigner alors `base_url` avec l'adresse d'un relais du distributeur, qui détient la clé et authentifie la caisse, et ne pas mettre `api_key_file`. Pour Worldline, la clé est un fichier de secret comme les autres :

```
(umask 027; cat > /etc/pili-pos/secrets/terminal.key)   # coller la clé, Entrée, puis Ctrl+D
chown root:pos /etc/pili-pos/secrets/terminal.key
pili-pos --check-config        # signale un champ manquant ou un fichier vide
systemctl restart pili-pos
```

Le seul trafic sortant ajouté est HTTPS (443) vers le nuage du fournisseur (types nuage), ou une connexion TCP vers le terminal sur le LAN (`ccv_opi` : la caisse ouvre le canal 0 vers `address` et écoute sur `device_port` pour le canal 1 ; aucun autre port entrant). Chaque fournisseur certifie lui-même l'intégration avant tout usage réel et délivre les identifiants nécessaires : la configuration d'un établissement n'est livrée qu'avec ces éléments. Les paiements par carte ne sont jamais arrondis, pour toute installation.

4.7 Paramètres de l'installation, journaux, réseau et comptes

Qui peut modifier quoi.

Paramètre	Valeur d'origine	Qui le modifie	Où
Identité et réseau : `pos_id`, `restaurant_*`, `fdm_*`, `printer_address`, `kitchen_printer_address`, `language`, `day_end_hour`, `table_count`, `tables`, `backup_dir`, `digital_ticket`, `hubrise`, `peppol`, `payment_terminal`	Gabarit d'usine (`CHANGE_ME_14CH` pour `pos_id`, `terminal_id` `POS1`)	Le technicien seul, en `root` sur la console `tty2`	`/etc/pili-pos/pos.json`, puis `systemctl restart pili-pos` (la configuration est lue au démarrage)
`device_id`	Adresse MAC, inscrite au premier démarrage	Personne (sauf changement de la carte réseau)	`/etc/pili-pos/pos.json`
`training_mode`	`false`	Valeur de départ : le technicien. Ensuite le Gérant, dans "Config", panneau "Mode Formation"	Le réglage de "Config" est en mémoire : il vaut jusqu'au redémarrage, qui revient à la valeur du fichier
`hubrise_enabled`	`false`	Valeur de départ : le technicien. Ensuite le Gérant, dans "Config", panneau "Intégrations"	Chaque changement est journalisé, ne s'efface pas, et le dernier l'emporte
`tables` (ou `table_count`)	Plan de 1 à `table_count`	Valeur de départ : le technicien. Ensuite le Gérant, dans "Config", panneau "Plan de salle"	Chaque changement est journalisé (`table_plan_log`), ne s'efface pas, et le dernier l'emporte
Langue d'affichage de l'interface	Clé `language`	Le Gérant, dans "Config", panneau "Langue"	Sans effet sur la langue des mentions imprimées
Employés et PIN	Aucun employé à l'origine (§4.4)	Le technicien (premiers comptes) puis le Gérant, dans "Config"	Table `users` de `pos.db`
Catalogue (catégories, articles, TVA)	Vide en production	Le technicien (`MANUEL_PROGRAMMATION_FR.md`), le Gérant dans "Config", panneau "Carte" (`EDITEUR_CARTE_FR.md`, journalisé), ou HubRise si celui-ci gère le menu	`pos.db`

Journaux. Le POS écrit un fichier par jour, `/var/lib/pili-pos/logs/pos.log.AAAA-MM-JJ`. Le journal du système (`journalctl -u pili-pos`) reçoit les messages de Cage et de l'interface. L'archive fiscale, la table `fiscal_events` de `pos.db`, n'est pas un journal : la base refuse toute modification ou suppression de ses lignes (déclencheurs SQL), elle est donc en ajout seul. Des sauvegardes de `pos.db` sont écrites dans `/var/lib/pili-pos/backups/` après chaque rapport Z et avant chaque mise à jour. La caisse garde les 7 sauvegardes les plus récentes qu'elle a faites ; elle ne supprime jamais les copies d'avant mise à jour ni les copies manuelles.

Réseau. Le terminal est raccordé en Ethernet (RJ45) ; NetworkManager gère la connexion et son adresse (DHCP, ou adresse fixe réservée sur le routeur, §4.3). Le trafic **sortant** est le suivant : le FDM sur le LAN (HTTPS 443, ou HTTP 80), l'imprimante sur le LAN (TCP 9100), HTTPS 443 vers `api.scaleway.com` si `digital_ticket` est configuré, HTTPS 443 vers `manager.hubrise.com` et `api.hubrise.com` si HubRise est activé, HTTPS 443 vers `api.storecove.com` si `peppol` est configuré, et une session RustDesk, à la demande d'un Gérant. Aucun port n'est ouvert en entrée.

Comptes du système. `root` (mot de passe aléatoire noté au premier démarrage, console `tty2` seulement), `pos` (compte du service pili-pos, sans connexion possible) et, sur l'image d'admission seulement, `spf` (console `tty2`, lecture seule de la base par `pos-db-ro`). Les employés du restaurant ne sont pas des comptes du système : ce sont les lignes de la table `users`, avec leur INSZ et leur PIN.

5. Vérification de l'intégrité du binaire

Le hash SHA-1 du binaire `pili-pos` certifié est consigné dans le formulaire SPF Finances. Vérifier :

```
sha1sum /opt/pili-pos/pili-pos
```

La valeur doit correspondre exactement à celle déclarée dans le formulaire. Si différente : binaire non-certifié, **NE PAS METTRE EN PRODUCTION**.

6. Mise à jour (procédure post-cert)

Une mise à jour suit la procédure officielle SPF Finances (`Manuel_Intro_ToRead_FR.md` §C) :

1. Le distributeur déclare la modification au SPF Finances :
   - Changelog signé
   - Affirmation qu'aucune autre modification n'a été apportée
   - Nouveau hash SHA-1
2. Délai de traitement SPF : 15 jours.
3. Après accord, le distributeur peut déployer sur les sites clients.

Le binaire d'une mise à jour n'est **jamais** installé à distance : ni par SSH (aucun serveur SSH), ni pendant une session RustDesk (qui donne accès à l'écran de la caisse, pas à un shell, et ne sert qu'à l'assistance, `deploy/REMOTE-OPS.md`). Il arrive sur place, sur une clé USB, par l'une des deux variantes ci-dessous. Dans les deux cas, le SHA-1 du binaire installé doit ensuite correspondre à la valeur déclarée (§5).

6.1 Variante A : remplacer le binaire seul (`update.sh`)

C'est la voie normale. La racine du système est en lecture seule : le script la remonte en écriture le temps de l'installation, puis la remet en lecture seule.

1. Préparer, sur le poste du distributeur, une clé USB (FAT32 ou ext4) qui contient le fichier `pili-pos` de la version déclarée. Noter le SHA-1 déclaré pour cette version.
2. Sur place, brancher la clé sur un port USB-A du terminal et un clavier USB sur l'autre. Ouvrir la console de maintenance (`Ctrl+Alt+F2`, login `root`, §4.1).
3. Repérer la clé, la monter en lecture seule et lancer le script (`/mnt` ne sert que de point de montage ; ne rien copier dans `/root`, qui est en lecture seule) :

   ```
   lsblk                                  # repérer la clé, par ex. sda1
   mount -o ro /dev/sda1 /mnt
   /opt/pili-pos/deploy/update.sh /mnt/pili-pos --sha1 <SHA-1 déclaré>
   umount /mnt
   ```

   Pour travailler sur une copie plutôt que sur la clé, la copier dans `/tmp` (en mémoire) ou `/var/tmp` (sur le volume chiffré) et passer ce chemin au script.

4. Le script refuse tout binaire dont le SHA-1 diffère de la valeur déclarée. Sinon il arrête le service, garde l'ancien binaire (`/opt/pili-pos/pili-pos.bak`), sauvegarde la base (`/var/lib/pili-pos/backups/pre-update-AAAAMMJJ-HHMMSS.db`), installe le nouveau binaire, applique les migrations de la base (`--init-db`, service arrêté), redémarre le service et affiche sa version.
5. Retirer la clé. Vérifier `sha1sum /opt/pili-pos/pili-pos`, puis faire un test de bout en bout abrégé (§4.6 : connexion, une vente, un rapport X) et contrôler que `fiscal_events` est intacte.

En cas de problème, `deploy/REMOTE-OPS.md` décrit le retour au binaire précédent (le binaire sauvegardé).

6.2 Variante B : réinstaller le système depuis l'image (entrée "Update")

À utiliser quand le système lui-même change, pas seulement le binaire : l'image kiosque complète de la version déclarée est écrite sur la clé (§2). Elle remplace l'ESP et la racine du terminal et **conserve** la partition de données (`/var` : base, sauvegardes, journaux, configuration).

1. Sauvegarder la base courante dans un dossier qui survit au redémarrage (pas `/tmp`, qui est en mémoire) :

   ```
   sqlite3 /var/lib/pili-pos/pos.db ".backup /var/lib/pili-pos/backups/avant-maj.db"
   ```

2. Brancher la clé et redémarrer le terminal ; au menu `systemd-boot`, choisir l'entrée **"Update"** (§3.2).
3. L'installateur affiche à l'écran (console `tty1`) un cadre qui nomme le disque cible et rappelle : "Le système sera remplacé ; les données (/var, base fiscale) sont CONSERVÉES.". Il demande : `Tapez OUI en majuscules pour confirmer :`. Taper `OUI` puis Entrée ; toute autre réponse annule.
4. L'installateur relève les mots de passe `root` et `spf` de l'ancien système, copie l'ESP, puis la racine, y réinscrit ces deux mots de passe (ils restent donc ceux du carnet du distributeur), répare la table de partitions et affiche : `Retirez la clé USB puis appuyez sur Entrée pour redémarrer…`. Retirer la clé, puis appuyer sur Entrée.
5. Au redémarrage, `pili-provision` ne s'exécute pas (`/var` est déjà monté) et la configuration est celle de la partition de données. Les migrations de la base s'appliquent au démarrage de pili-pos.
6. Vérifier `sha1sum /opt/pili-pos/pili-pos` (§5), faire le test abrégé de la variante A et contrôler que `fiscal_events` est intacte.

Le choix **"Install"** du même menu efface au contraire le disque interne : il ne sert qu'à une installation neuve (§3.3), jamais à une mise à jour.

7. Annexes

7.1 Liste des numéros INSZ de test

Les comptes de l'image d'admission (`src-tauri/seed_spf_test.sql`) : le personnel du jeu de test SPF (`UC_TestSetup_20240806.md`) et un compte par rôle.

Rôle	Nom	INSZ	PIN
Serveur	John (barman)	75061189702	4444
Serveur	Elisa (chef de salle)	84022899837	2222
Serveur	Sammy (sommelier)	80113078968	5555
Gérant	Gerant Test	94060242576	1111
Gérant	Technicien SPF	00000000097	0097
Apprenti	Apprenti Test	99012345667	3333
Robot (automate, identité système, sans connexion)	Robot / Kiosk	00000000029	-

À utiliser exclusivement lors de la phase de test pré-soumission.

7.2 Numéros pos_id de test

CP0S0031234567 - le posId de la configuration de test SPF Finances (UC_TestSetup_20240806), autorisé sur le simulateur module2 de la machine de test.

7.3 FDM

- **FDM module2** (PRACSYS, certifié SCE 2.0) : **référence retenue.** GraphQL sur le LAN en HTTPS (port 443, certificat client mTLS fourni par module2), configuration retenue ; HTTP (port 80) reste possible. Les trois fichiers de module2 (`ca.crt`, `client.crt`, `client.key`, publiés sur manual.module2.be, identiques pour tous ses FDM et son simulateur) vont dans `/etc/pili-pos/fdm/`, la clé en droits 600, et sont désignés par `fdm_ca_file`, `fdm_client_cert_file` et `fdm_client_key_file`. Le POS vérifie que le certificat du FDM est signé par cette autorité et en cours de validité (il ne porte pas de nom d'hôte, le FDM étant joint par son IP). Chaque FDM exige son jeton Bearer (lu sur sa page de paramètres ou dans le portail module2), à placer dans le fichier désigné par `fdm_token_file`. Le posId du POS doit être autorisé dans les paramètres du FDM.
- **Simulateur en ligne module2** : `https://fdm.module2.be/graphql/` (mêmes certificats) ou `http://fdm.module2.be/graphql/`, même jeton Bearer, utilisé pour les tests pré-certification et comme FDM de la machine de test à l'Intake (SCHEMA_ASSEMBLAGE_FR.md §7).
- Le POS parle le protocole GraphQL standard du SPF, sans extension propriétaire : tout FDM certifié SCE 2.0 joignable en HTTP sur le LAN convient (jeton Bearer facultatif selon le modèle).

7.4 Contact distributeur

Bruno Moyse / nuagemagique.dev - Email : cloud@nuagemagique.dev - Bruxelles, Belgique

7.5 Dépannage : réponse du FDM en attente

Symptôme : toute vente, tout rapport ou tout pointage est refusé avec "La caisse attend la réponse de la boîte noire pour l'opération précédente" (code `FDM_ANSWER_PENDING`, dans les journaux avec le label et le numéro de ticket concernés).

Cause : la dernière requête a été envoyée au FDM mais sa réponse n'est pas revenue. La requête est conservée telle quelle (table `pending_signs`, ligne au statut `open`) et renvoyée à l'identique, par la tâche de fond toutes les 60 secondes et avant toute autre signature. Le FDM, s'il l'avait signée, renvoie sa réponse (`DUPLICATE_REQUEST`) et l'événement est archivé une seule fois. Il n'y a pas d'abandon automatique : la signature reste bloquée tant que le FDM n'a pas répondu.

Procédure :

1. Vérifier le câble réseau, l'alimentation et le voyant du FDM (bandeau de la caisse, statut du FDM).
2. Attendre une minute : le renvoi automatique débloque la caisse dès que le FDM répond.
3. Si le blocage dure, vérifier l'état du FDM (statut, erreurs, zone tampon pleine ou FDM verrouillé) et contacter le distributeur ou le technicien.
4. **Ne jamais supprimer ni modifier la ligne `pending_signs`** (les déclencheurs l'interdisent) et ne jamais refaire la vente à la main.
5. Si le FDM répond par un refus, la ligne passe seule à `refused`, son numéro est libre et la caisse repart.

Remarque : un FDM qui aurait perdu sa mémoire des doublons signerait le renvoi comme un nouvel événement. Les compteurs le détectent et la caisse le journalise ; le technicien le constate à l'écart de numérotation.